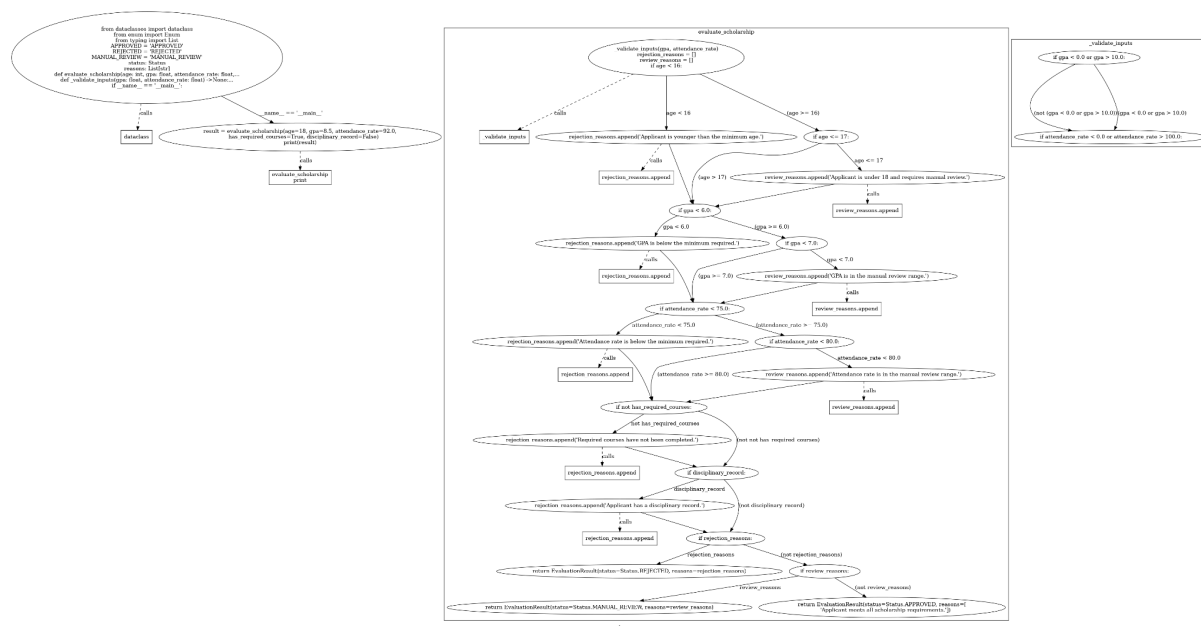


Relatório da APS

1. Escolha Tecnológica

Python e pytest foram escolhidos pela simplicidade, legibilidade e forte suporte no ecossistema. O pytest oferece recursos robustos como parametrização e integração com cobertura via pytest-cov, permitindo análise direta com um único comando. A biblioteca staticfg foi utilizada para gerar o CFG, possibilitando identificar decisões e caminhos do código para os testes estruturais.

As dependências utilizadas foram: staticfg==0.9.5 (CFG), graphviz==0.21 (visualização), pytest==9.0.2 (testes) e pytest-cov==7.1.0 (cobertura). As demais bibliotecas (astor, pluggy, iniconfig, packaging, Pygments) são dependências transitivas dessas ferramentas.



2. Classes de Equivalência Identificadas

A partir da análise do código-fonte e do CFG gerado, foram identificadas as seguintes classes de equivalência para cada parâmetro de entrada:

Variável	REJECTED	MANUAL_REVIEW	APPROVED
age	< 16	16–17	≥ 18
gpa	< 6.0 < 0 ou > 10 (erro)	6.0–6.9	≥ 7.0
attendance_rate	< 75.0 < 0 ou > 100 (erro)	75.0–79.9	≥ 80.0
has_required_courses	False	—	True
disciplinary_record	True	—	False

As classes de equivalência não são independentes entre si. A decisão final segue a seguinte prioridade: (1) qualquer condição de rejeição resulta em REJECTED; (2) caso contrário, qualquer condição de revisão resulta em MANUAL_REVIEW; (3) se nenhuma condição for acionada, o resultado é APPROVED.

3. Valores Limite Considerados

Foram testados pontos críticos de transição: Idade: 15, 16, 17, 18. GPA: 6.0, 7.0 Frequência: 75.0, 79.9, 80.0 Esses valores garantem a verificação correta dos operadores (<, <=) e evitam erros de fronteira.

4. Decisões e Branches Relevantes do Código

Com base no CFG, foram mapeadas todas as decisões relevantes do código e os testes que as exercitam:

Decisão no Código	Teste(s) que a cobrem
age < 16	test_evaluate_scholarship_rejected_age, test_evaluate_scholarship_rejected_all_factors
age <= 17	test_evaluate_scholarship_manual_review_age, test_boundary_age_18, test_evaluate_scholarship_manual_review_age_gpa, test_evaluate_scholarship_manual_review_age_gpa_attendance
gpa < 6.0	test_rejection_overrides_manual_review, test_evaluate_scholarship_rejected_all_factors
gpa < 7.0 (revisão)	test_evaluate_scholarship_manual_review_attendance, test_boundary_gpa_6, test_evaluate_scholarship_manual_review_age_gpa
attendance_rate < 75.0	test_rejected_attendance_only, test_evaluate_scholarship_rejected_all_factors
attendance_rate < 80.0 (revisão)	test_evaluate_scholarship_manual_review_attendance, test_boundary_attendance_75
not has_required_courses	test_evaluate_scholarship_rejected_required_courses, test_evaluate_scholarship_rejected_all_factors
disciplinary_record	test_evaluate_scholarship_rejected_disciplinary_record, test_evaluate_scholarship_rejected_all_factors
GPA inválido (< 0 ou > 10)	test_evaluate_scholarship_invalid_gpa
Frequência inválida (< 0 ou > 100)	test_evaluate_scholarship_invalid_attendance_rate
Rejeição sobrepõe revisão	test_rejection_overrides_manual_review

A suíte exercita todos os branches dessas decisões, diferentes combinações de entrada e os quatro caminhos de execução distintos: aprovação, revisão manual, rejeição e lançamento de exceção.

5. Análise de Adequação da Suíte

5.1 Combinação de Pares de Fatores

Para complementar a cobertura funcional e estrutural, a suíte aplica a técnica de combinação de pares de fatores (pairwise testing). Essa abordagem garante que todas as interações possíveis entre quaisquer dois parâmetros de entrada sejam testadas pelo menos uma vez, aumentando a probabilidade de detectar falhas causadas por combinações específicas, sem gerar a explosão combinatória de testar todas as combinações possíveis.

5.2 Resumo de Adequação

Critério	Status
Todas as classes de equivalência cobertas	✓ Sim
Valores limite críticos validados	✓ Sim
Todos os branches do CFG exercitados	✓ Sim
Caminhos de execução distintos testados	✓ Sim (aprovação, revisão, rejeição, exceção)
Entradas inválidas cobertas	✓ Sim (GPA e frequência)
Conflitos entre regras testados	✓ Sim (rejeição vs. revisão)
Cobertura de código obtida	~100% (exceto bloco <code>__main__</code>)

O bloco `if __name__ == '__main__':` é excluído da métrica de cobertura por ser um bloco de execução direta (demonstração), não pertencente à lógica testável do sistema. Sua exclusão é padrão em análises de cobertura com `pytest-cov`.

5.2 Limitações da Suíte

Apesar da alta cobertura e da abordagem estruturada, a suíte possui as seguintes limitações: A suíte não cobre todas as combinações possíveis devido à explosão combinatória, depende da interpretação da lógica implícita no código (sem especificação formal), não avalia aspectos não funcionais e pode não capturar todas as interações complexas entre variáveis em cenários combinados.

5.3 Por que Alta Cobertura Não Garante Ausência de Defeitos

Cobertura de código indica apenas quais trechos foram executados, não garantindo correção do comportamento. Mesmo com alta cobertura, podem existir falhas como regras de negócio incorretas, erros em limites, interações não previstas entre variáveis e problemas na própria especificação. Assim, cobertura estrutural é necessária, mas não suficiente para garantir qualidade.

6. Conclusão

A suíte desenvolvida combina técnicas funcionais (classes de equivalência e análise de valor limite) com técnicas estruturais (análise de CFG e cobertura de branches), resultando em uma abordagem robusta e eficiente para validação do Scholarship Eligibility Evaluator. Os requisitos mínimos da atividade foram integralmente atendidos: a suíte inclui casos de APPROVED, MANUAL_REVIEW, múltiplos casos de REJECTED por motivos distintos, entradas inválidas e valores de fronteira, além de testes específicos para cada decisão relevante do código. A estratégia adotada equilibra qualidade e custo de testes,

priorizando a cobertura dos comportamentos esperados pelo sistema sem incorrer em redundância desnecessária.